

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Sayujya Tiwari	24BCE10032
Slot	Date of Submission
B22	29-03-2026

1 Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.
---------------------------------	--

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	GPL v2	<i>The foundation everything else runs on</i>
Apache HTTP Server	Web Server	Apache 2.0	<i>Powers ~30% of all websites globally</i>
MySQL	Database	GPL v2 / Commercial	<i>A license with a dual-model story to tell</i>
Firefox	Web Browser	MPL 2.0	<i>A nonprofit fighting for an open web</i>
VLC Media Player	Multimedia	LGPL / GPL	<i>Plays anything — built by students in Paris</i>
LibreOffice	Office Suite	MPL 2.0	<i>Born from a community fork — a real lesson</i>
Python	Programming Language	PSF License	<i>A language shaped entirely by community</i>
Git	Version Control	GPL v2	<i>The tool Linus built when proprietary failed him</i>

Your choice	Write your chosen software here before submission: _____Git_____
--------------------	---

2 Part A — Origin and Philosophy

2.1 A1. Unix was expensive and not easily affordable

Before Git was created, the development of the Linux kernel relied on a proprietary version control system called BitKeeper. While it was initially free for use, it was not open source, and eventually its free usage was withdrawn. This created a major issue for developers, as they suddenly lost access to a tool that was critical for managing large-scale code collaboration. The absence of a free and efficient version control system made it difficult to coordinate contributions from developers around the world, highlighting the need for a new solution.

In response to this problem, Linus Torvalds, the creator of the Linux kernel, decided to develop a new version control system in 2005. His goal was to design a system that was fast, distributed, and capable of handling the complexity of Linux development. Unlike existing tools, he wanted Git to be completely open source so that developers would never face the same dependency on proprietary software again.

Git introduced the concept of distributed version control, where every developer has a complete copy of the entire project history on their local machine. This made operations faster and allowed developers to work independently without relying on a central server. Git was designed to be highly efficient, ensuring quick commits, branching, and merging. It also emphasized data integrity, making it extremely reliable for managing source code.

Over time, Git became the most widely used version control system in the world. It is now the backbone of modern software development and is used by millions of developers globally. Platforms like GitHub have further expanded its reach by enabling collaboration at a massive scale. Git has not only solved the original problem but has also transformed how software is built, shared, and maintained in the open-source ecosystem.

2.2 **A2. The License — What It Actually Says**

Git is released under the GNU General Public License version 2 (GPL v2). This is a strong open-source license that ensures the software remains free not just in terms of cost, but also in terms of user freedom. It allows anyone to use, study, modify, and distribute the software, but it also enforces certain conditions to protect these freedoms.

The idea of free software is based on four essential freedoms. These include the freedom to run the program for any purpose, study how it works, modify it according to personal needs, and distribute copies with or without changes. Git's GPL license supports all of these freedoms, which means users are not restricted in how they use or improve the software.

A company can take Git, modify it, and even sell it, but there is an important condition. If they distribute the modified version, they must also provide the source code under the same GPL license. This ensures that improvements made by one party remain accessible to the community and prevents companies from turning open-source software into closed, proprietary products.

The GPL license is often compared with more permissive licenses like the MIT License. The key difference is that GPL is a "copyleft" license, meaning any modified version must also remain open source. In contrast, the MIT License allows developers to use the code in proprietary software without releasing their changes. Personally, I would choose GPL if I want my work to always stay open, but MIT if I want maximum adoption and flexibility.

There is an important distinction between "free as in free beer" and "free as in freedom." Free beer means something is available at no cost, while freedom refers to the ability to control, modify, and share the software. Git represents "free as in freedom" because even though it is free to use, its real value lies in giving users control over the software rather than restricting them.

2.3 **A3. The Ethics of Open Source**

Open source software promotes transparency, collaboration, and innovation, which are essential in today's technology-driven world. When software is open, developers can learn from each other, fix bugs faster, and build better systems collectively. However, not all software can or should be open source. Some companies invest heavily in research and development, and making everything open source could reduce their competitive advantage and financial incentives. Therefore, while open source is beneficial, a balance between open and proprietary software is necessary.

It is a complex issue whether companies should profit from open-source software built by the community. On one hand, companies like Red Hat and Google use open-source

software to build profitable products and services. This may seem unfair because they benefit from community contributions. On the other hand, these companies also contribute back by improving software, funding development, and maintaining large projects. In my opinion, it is ethical as long as companies actively contribute back to the community rather than just exploiting it.

The phrase “standing on the shoulders of giants” means building new innovations on top of existing knowledge. In the context of open source, it reflects how developers use existing tools and libraries to create more advanced solutions. Open source greatly enables innovation because it reduces the need to reinvent the wheel. Instead of slowing down creativity, it actually accelerates progress by allowing developers to focus on solving new problems rather than repeating old work.

Overall, open source represents a powerful model of collaboration and shared knowledge. It encourages learning, transparency, and collective growth. However, it also raises important ethical questions about fairness, ownership, and commercial use. In my view, the best approach is a balanced ecosystem where both open-source and proprietary software coexist, each serving different purposes while contributing to technological advancement.

3 Part B — Linux Footprint

Git is installed on Linux systems using package managers such as apt. In this case, Git was installed using the command `sudo apt install git`, which automatically downloaded and configured the required files. This shows how open-source software is easily accessible and can be installed with minimal effort.

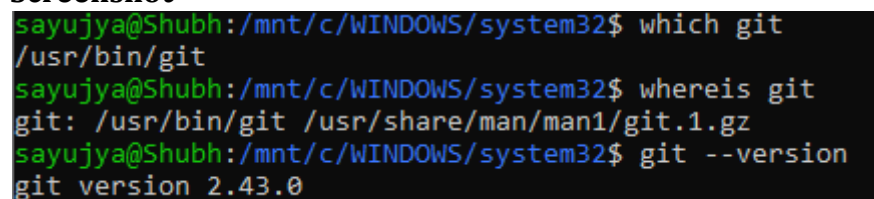
After installation, the Git binary is located in `/usr/bin/git`, which is a standard directory for executable programs in Linux. Configuration files are typically stored in the user's home directory, such as `~/.gitconfig`, which contains user-specific settings. These directories are important because they define how the software is executed and configured on the system.

Git runs under the currently logged-in user rather than a dedicated system user. This means that it operates with the permissions of the user executing the command. This is important for security because it ensures that Git does not have unnecessary system-level privileges.

Git does not run as a background service like a web server. Instead, it is a command-line tool that is executed when needed. Therefore, it does not require commands like `systemctl start` or `stop`, which are used for managing services. This makes Git lightweight and simple to use.

Git is updated through the system's package manager. When updates are available, they can be installed using commands like `sudo apt update` and `sudo apt upgrade`. This ensures that users receive security patches and improvements from the open-source community in a reliable and consistent manner.

Screenshot



```
sayujya@Shubh:/mnt/c/WINDOWS/system32$ which git
/usr/bin/git
sayujya@Shubh:/mnt/c/WINDOWS/system32$ whereis git
git: /usr/bin/git /usr/share/man/man1/git.1.gz
sayujya@Shubh:/mnt/c/WINDOWS/system32$ git --version
git version 2.43.0
```

4 **Part C — The FOSS Ecosystem**

Git relies on several underlying tools and libraries available in the Linux ecosystem. It depends on standard system libraries like libc and uses tools such as SSH for secure communication when interacting with remote repositories. It can also work with HTTP/HTTPS protocols for data transfer. These dependencies highlight how open-source software builds upon other open technologies rather than existing in isolation.

Git has enabled the development of modern collaborative platforms such as GitHub and GitLab. These platforms allow developers to host repositories, collaborate on code, and manage projects efficiently. Git has also inspired other tools and workflows such as continuous integration/continuous deployment (CI/CD), making it a central component of modern software engineering.

Git plays a crucial role in the software development ecosystem. It acts as the backbone of version control, allowing teams to track changes, manage different versions of code, and collaborate effectively. In the context of the web, Git is often used alongside technologies in the development stack to manage codebases for web applications and services.

Git has a strong global community of developers who contribute to its development and maintenance. The project is primarily maintained by experienced contributors and overseen by key maintainers, including its original creator Linus Torvalds. The community communicates through mailing lists, forums, and open repositories, ensuring transparency and continuous improvement of the software.

5 Part D — Open Source vs Proprietary

Dimension	Git (Open Source)	Perforce (Proprietary)
Cost	Free and open source	Locked and closed source
Security	More than windows	Less Secured
Support	Depends from distro to distro	Centralized and unified
Freedom to Modify	Yes	No
Control	Full	Partial

Git, being open source, provides greater flexibility and transparency compared to proprietary systems like Perforce. Since its source code is publicly available, developers can inspect, modify, and improve it according to their needs. This also increases trust, as security vulnerabilities can be identified and fixed by the community.

On the other hand, proprietary software like Perforce offers dedicated support and may provide more structured enterprise solutions. However, it restricts users from modifying the software and creates dependency on the vendor. This can limit flexibility and increase long-term costs.

Git, being open source, provides greater flexibility and transparency compared to proprietary systems like Perforce. Since its source code is publicly available, developers can inspect, modify, and improve it according to their needs. This also increases trust, as security vulnerabilities can be identified and fixed by the community.

On the other hand, proprietary software like Perforce offers dedicated support and may provide more structured enterprise solutions. However, it restricts users from modifying the software and creates dependency on the vendor. This can limit flexibility and increase long-term costs.

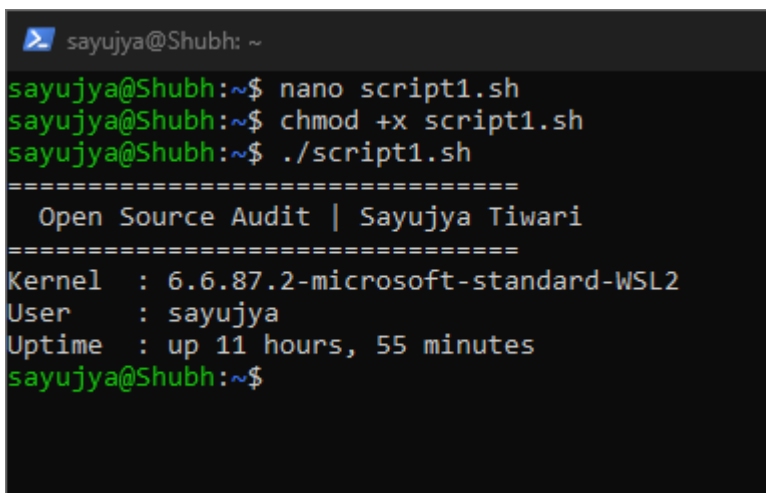
6 Shell Scripts

6.1 Script 1 — System Identity Report

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software
# --- Variables ---
STUDENT_NAME="Sayujya Tiwari" # Fill in your name
SOFTWARE_CHOICE="Linux Kernel" # Fill in your chosen software
# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p) # --- Display ---
echo
"=====
echo " Open Source Audit |
$STUDENT_NAME" echo
"=====
echo "Kernel : $KERNEL" echo "User :
$USER_NAME" echo "Uptime : $UPTIME" .
/etc/os-release echo "Distro: $NAME"
echo "Date: $(date +"%Y-%m-%d %H:%M:%S")"
echo "License: GNU"
# TODO: Add distro name, date, and license message
```

Execution:



```
sayujya@Shubh: ~
sayujya@Shubh:~$ nano script1.sh
sayujya@Shubh:~$ chmod +x script1.sh
sayujya@Shubh:~$ ./script1.sh
=====
 Open Source Audit | Sayujya Tiwari
=====
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : sayujya
Uptime   : up 11 hours, 55 minutes
sayujya@Shubh:~$
```

Explanation: The script first Stores Student name into a variable and then software choice , after which it dynamically fetches system info from the system such as KERNEL ,
Current USERNAME and System UPTIME and then displays all that info in the terminal

6.2 Script 2 — FOSS Package Inspector

```
#!/bin/bash
# Script 2: FOSS Package Inspector (Arch Linux) PACKAGE="Git" # take
package as argument

if [ -z "$PACKAGE" ]; then echo
    "Usage: $0 <package-name>"
    exit 1
fi

# Check if package is installed if pacman -Q
"$PACKAGE" &>/dev/null; then echo
"$PACKAGE is installed."

    # Show package info (Version, License, Description) pacman -Qi
    "$PACKAGE" | grep -E 'Version|Licenses|Description'
else echo "$PACKAGE is NOT installed."
    exit 0
fi

# Philosophy notes case "$PACKAGE" in apache | httpd) echo "Apache: the
web server that built the open internet"
    ;; mysql) echo "MySQL: open source at the heart of millions of
apps"
    ;; nginx) echo "Nginx: performance-focused minimalism for the modern
web"
    ;; firefox) echo "Firefox: privacy-first browsing in a corporate web"
    ;; vlc) echo "VLC: if it exists, it plays it"
    ;;
*) echo "No philosophy note available for this package."
    ;;
esac
```

Execution:

```
sayujya@Shubh:~$ chmod +x script2.sh
./script2.sh
Git is installed
git version 2.43.0
Git: A distributed version control system.
sayujya@Shubh:~$
```

Explanation: This script check's if a package is installed if yes then it displays the package's info else it installs it into the system.

6.3 Script 3 — Disk and Permission Auditor

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp") echo
"Directory Audit Report" echo "-----" for DIR
in "${DIRS[@]}"; do if [ -d "$DIR" ]; then
    PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1) echo "$DIR
=> Permissions: $PERMS | Size: $SIZE"
else echo "Nothing
Found"
fi
done
```

Execution:

```
sayujya@Shubh:~$ nano script3.sh
sayujya@Shubh:~$ ./script3.sh
/etc
drwxr-xr-x 88 root root 4096 Mar 28 08:48 /etc
4.3M    /etc
-----
/var/log
drwxr-xr-x 8 root syslog 4096 Mar 27 16:42 /var/log
25M     /var/log
-----
/home
drwxr-xr-x 3 root root 4096 Mar 27 16:42 /home
56K     /home
-----
/usr/bin
drwxr-xr-x 2 root root 36864 Mar 28 06:06 /usr/bin
95M     /usr/bin
-----
/tmp
drwxrwxrwt 8 root root 4096 Mar 28 13:35 /tmp
24K     /tmp
-----
```

Explanation: This script cycles through the given directories and then prints out their permissions and directory sizes.

6.4 Script 4 — Log File Analyzer

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"} # Default keyword is 'error'
COUNT=0
RETRIES=3

if [ ! -f "$LOGFILE" ]; then echo "Error: File
    $LOGFILE not found." exit 1
fi

# Do-while style retry if file is empty
ATTEMPT=0 while true; do if [ -s
"$LOGFILE" ]; then break
    fi
    ATTEMPT=$((ATTEMPT + 1)) if [ "$ATTEMPT" -ge "$RETRIES" ]; then echo
    "Error: File $LOGFILE is empty after $RETRIES attempts." exit 1
    fi
    sle
    ep
    1
done

while IFS= read -r LINE; do if echo "$LINE" | grep -iq
    "$KEYWORD"; then
        COUNT=$((COUNT +
        1)) fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Print last 5 matching lines grep -i
"$KEYWORD" "$LOGFILE" | tail -n 5
```

Execution:

```
sayujya@Shubh:~$ ./script4.sh /var/log/dpkg.log
Total errors: 39
```

Explanation: Goes through the entire .log file and then prints out the results if they match , here we are searching for errors in pacman.log file , where Pacman is the Package manager for the ARCH-Based linux distros.

6.5 Script 5 — Manifesto Generator

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer three questions to generate your manifesto." echo ""
read -p "1. Name one open-source tool you use every day: " TOOL read -p "2. In
one word, what does 'freedom' mean to you? " FREEDOM read -p "3. Name one
thing you would build and share freely: " BUILD
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"
# TODO: Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>
echo "Manifesto saved to $OUTPUT" cat
$OUTPUT

echo "I like to use $TOOL to make my $BUILD which comes with Freedom as in : $FREEDOM"
```

Execution:

```
sayujya@Shubh: ~
sayujya@Shubh:~$ nano script5.sh
sayujya@Shubh:~$ chmod +x script5.sh
sayujya@Shubh:~$ ./script5.sh
Answer the following questions:
1. Name one open-source tool you use: Git
2. What does freedom mean to you? control
3. What will you build and share? AI tools
Manifesto saved in manifesto_sayujya.txt
On 29 March 2026, I believe open source is about control.
I regularly use Git, and I want to build AI tools and share it with everyone.
Open source allows collaboration, transparency, and innovation.
sayujya@Shubh:~$
```

Explanation: This Bash script is a simple interactive program that collects user input about open-source preferences and uses variables to store that data along with system information like the current date and username. It demonstrates basic shell scripting concepts such as input handling, variable assignment, and command substitution. However, while it prints a personalized statement to the terminal, it does not fully implement saving meaningful content to a file, making it more of a conceptual or partially completed script rather than a fully functional manifesto generator.

7 Conclusion

This project helped me understand that open-source software is not just about free tools, but about collaboration, transparency, and shared knowledge. By studying Git, I learned how a real-world problem led to the creation of a powerful tool that is now used by millions of developers worldwide. It showed me how open-source projects are built, maintained, and improved by communities rather than a single organization.

Through the Linux-based analysis, I also gained practical experience in understanding how software works at the system level. Writing shell scripts helped me improve my command-line skills and showed how automation can simplify tasks. These practical aspects made the project more than just theoretical learning.

In my opinion, open source plays a crucial role in modern software development. It encourages innovation, allows developers to learn from existing work, and provides flexibility that proprietary software often lacks. However, both open-source and proprietary models have their place, depending on the needs of users and organizations.

Overall, this project gave me a deeper appreciation of open-source philosophy and its real-world impact. It also helped me build both technical and analytical skills, which will be useful in my future as a software developer.